David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

# Advanced CNN Architectures and Gated Models

① Multi-Channel CNNs
② CNN Model Comparisons
③ Vertical Gating
④ Batch Normalization
⑤ 1×1 Convolutions
⑥ Very Deep CNNs

# The Role of Multi-Channel in CNNs

- **Multi-channel input** allows CNNs to capture different feature representations (e.g., RGB channels in images or different embeddings in NLP).
- **Why multi-channel?** It enhances feature extraction by considering multiple perspectives simultaneously.
- In NLP, multi-channel embeddings (e.g., static vs. contextual word embeddings) provide richer representations.
- **Final goal:** A more robust and expressive feature representation for downstream tasks.

# What Comes After Feature Representation?

- **Feature Representation → Classifier Input:**
  - The extracted features are passed through a **softmax classifier** (or another classification layer) for prediction.
- **CNN as a Bag of Words:**
  - CNN essentially acts as a feature extractor, learning representations akin to a **bag-of-words** model in NLP.
- **Final Step:**
  - Softmax normalizes outputs into probabilities, enabling classification.
  - Other tasks: Feature vectors can also be used in **clustering, retrieval, or further deep learning layers**.

# Model Comparison – Bag of Vectors

- **Bag of Vectors: A Surprisingly Good Baseline**
  - Simple yet effective for **basic classification tasks**.
  - Treats feature vectors independently, without considering order or context.
  - Works well in many NLP and vision tasks with minimal computation.
- **Enhancement with ReLU Layers**
  - Adding **a few ReLU layers** improves non-linearity and model capacity.
  - Enables the model to learn **more complex representations** from the input vectors.
  - Still remains computationally efficient while boosting performance.

# Model Comparison – Window Model

- **Good for Single Word Classification**
  - Extracts **local context** for classification tasks.
  - Ideal for problems where **wide context is unnecessary**.
- **Best for Specific NLP Tasks**
  - **Part-of-Speech (POS) Tagging** – Focuses on word-level context.
  - **Named Entity Recognition (NER)** – Identifies entities based on immediate surroundings.
- **Trade-off:**
  - Limited **context awareness** compared to sequence models.
  - Works best when global dependencies are not crucial.

# Model Comparison – CNNs

- **Strengths:**
  - **Good for classification** – Captures local features effectively.
  - **Efficient & versatile** – Works well across text, image, and speech tasks.
  - **Easily parallelizable on GPUs** – Enables faster training.
- **Challenges:**
  - **Requires zero-padding** for handling variable-length inputs.
  - **Hard to interpret** – Features are learned through convolutions, making explainability difficult.

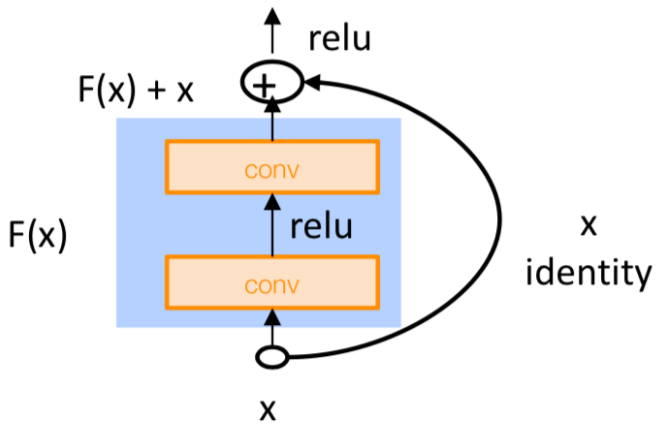
# Model Comparison – Recurrent Neural Networks (RNNs)

- **Strengths:**
  - **Cognitively plausible** – Processes data sequentially (left to right in text).
  - **Great for sequence tagging & classification** – Useful in speech and NLP tasks.
  - **Powerful for language modeling** – Captures temporal dependencies.
  - **Works well with attention mechanisms** – Improves long-range context learning.
- **Challenges:**
  - **Not optimal for classification** when using only the final state.
  - **Much slower than CNNs** due to sequential computation.
  - **Difficult to parallelize** – Training time increases with longer sequences.

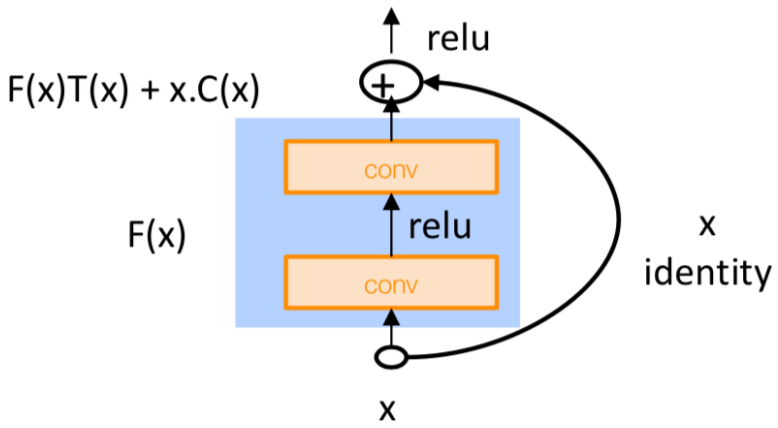
# Gated Units Used Vertically

- **Generalizing Gating Mechanisms:**
  - Gating and skipping mechanisms (as seen in **LSTMs & GRUs**) are now used across various deep learning architectures.
  - These mechanisms **regulate information flow**, helping networks retain important features while skipping redundant ones.
- **Vertical Gating:**
  - Gates can be applied **not just across time (RNNs), but also across depth** in deep networks.
  - Helps in efficient **gradient propagation** through deep architectures.

# Gated Units Used Vertically




Residual block
(He et al. ECCV 2016)




Highway block
(Srivistava et al. NeurIPS 2015)

Note: pad **x** for conv so same size when add them

# Why Vertical Gating Matters?

- **Key Idea: Candidate Update + Shortcut Connection**
  - Deep networks suffer from **vanishing gradients** without proper information flow.
  - Summing candidate updates with **shortcut (residual) connections** is **critical for deep networks**.
- **Real-World Applications:**
  - **Residual Networks (ResNets)** – Uses vertical gating via residual connections.
  - **Highway Networks** – Explicitly gates information between layers.
  - **Transformers & Modern Architectures** – Utilize gating for efficient deep learning.
- **Impact:**
  - Enables **very deep networks** to learn efficiently.
  - Improves **training stability** and **representation learning**.

# Batch Normalization – What It Does

- **Commonly used in CNNs** to stabilize and accelerate training.
- **How it works:**
  - Normalizes the output of a convolutional layer across a batch.
  - Ensures activations have **zero mean** and **unit variance**.
  - Introduces trainable **scale (γ) and shift (β) parameters** to restore flexibility.
- **Key Benefits:**
  - Reduces **internal covariate shift**, making training more stable.
  - Helps prevent vanishing/exploding gradients in deep networks.

# Why Batch Normalization Helps?

- **Less sensitivity to parameter initialization:**
  - Automatically **rescales outputs**, reducing reliance on careful weight initialization.
- **Easier hyperparameter tuning:**
  - Learning rates can be set **more aggressively** without causing instability.
  - Reduces the need for **very small learning rates** to avoid divergence.
- **Additional Benefits:**
  - Acts as a form of **regularization**, reducing dependence on dropout.
  - Helps **speed up convergence** by allowing higher learning rates.

# 1×1 Convolutions

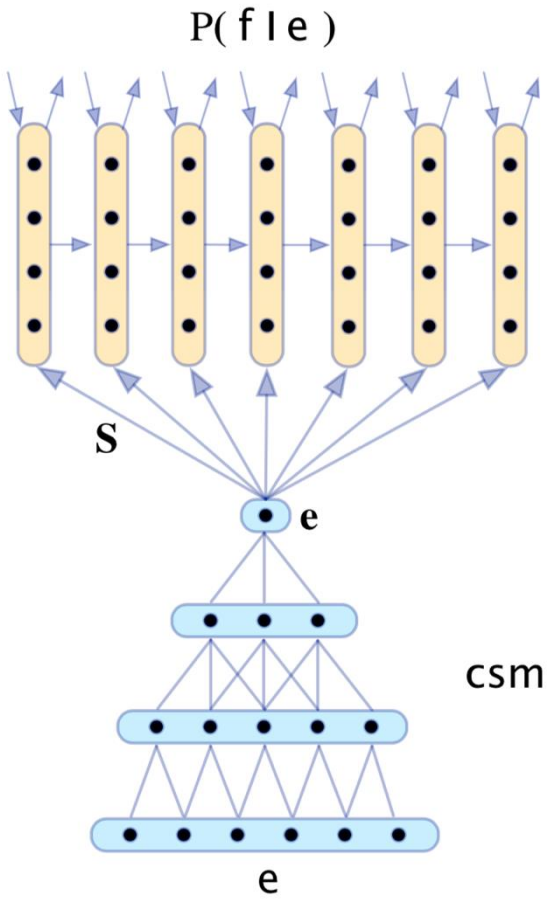
- **What Are 1×1 Convolutions?**
  - **Also known as Network-in-Network (NiN) connections.**
  - A **1×1 convolution** is a convolutional kernel with **kernel_size = 1**.
  - **Key Property:**
    - Acts as a **fully connected linear layer** across channels.
    - Enables **learning channel-wise transformations** without affecting spatial dimensions.

# Why Use 1×1 Convolutions?

- **Efficient Dimensionality Reduction:**
  - Maps **many channels → fewer channels**, reducing computation.
- **Adds More Network Depth:**
  - Introduces **additional layers** with very few extra parameters.
- **Better Than Fully Connected (FC) Layers:**
  - FC layers introduce a large number of parameters, whereas **1×1 convolutions keep it lightweight**.
- **Common Applications:**
  - Used in **GoogleNet (Inception modules)** for efficient feature reduction.
  - Helps in **bottleneck layers** in architectures like ResNet.

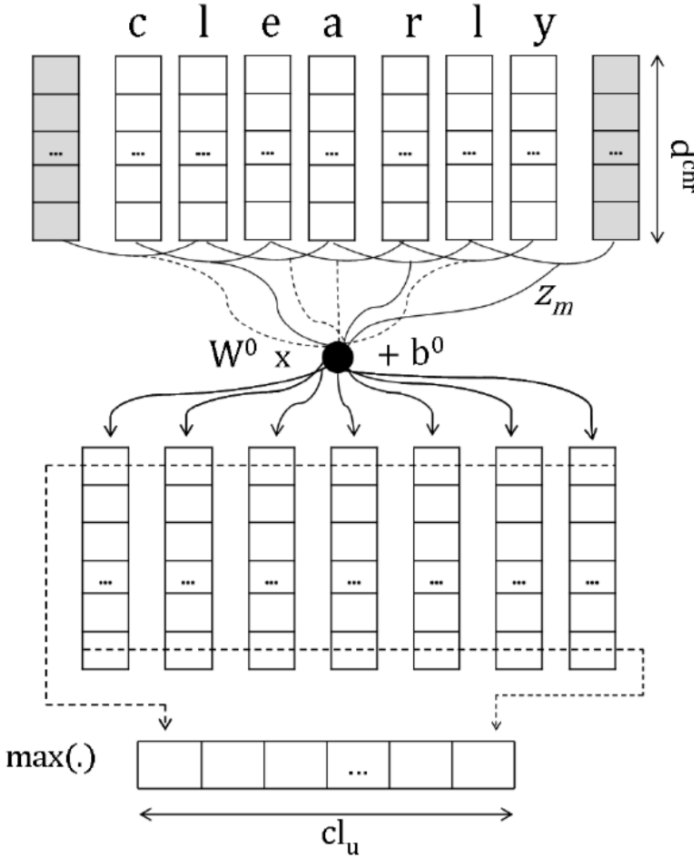
# Application: NMT

- **Early Neural Machine Translation (NMT) Models:**
  - One of the **earliest efforts** in NMT leveraged **1×1 convolutions** for efficient encoding.
  - CNNs were used for the **encoder**, processing input sequences in parallel.
  - RNNs used for the **decoder**, generating translated sequences step by step.
- **Why Use 1×1 Convolutions?**
  - **Efficient feature transformation** across channels in the encoding stage.
  - Helps **reduce dimensionality** while retaining important information.
  - Enables **faster processing** compared to traditional RNN-based encoders.
- **Impact on NMT:**
  - Inspired later **ConvS2S (Fully Convolutional Seq2Seq Models)** for translation.
  - **CNN-based encoders** could compete with RNNs in sequence modeling.



# Application: Part-of-Speech (PoS) Tagging

- **Learning Part-of-Speech (PoS) with CNNs:**
  - **1×1 convolutions** are used to process **character-level embeddings** of words.
  - CNNs **learn word representations** by applying convolutions over character sequences.
- **PoS Tagging Pipeline:**
  - **Character-level CNN** → Generates word embeddings from character sequences.
  - **Fixed window of word embeddings** → Captures local context for tagging.
  - **PoS classification** → A final classifier assigns PoS labels based on embeddings.
- **Why Use 1×1 Convolutions?**
  - Efficiently transforms character-level features into **compact word representations**.
  - Reduces computational complexity while **preserving linguistic information**.
  - Works well for morphologically rich languages where character-level cues are important.

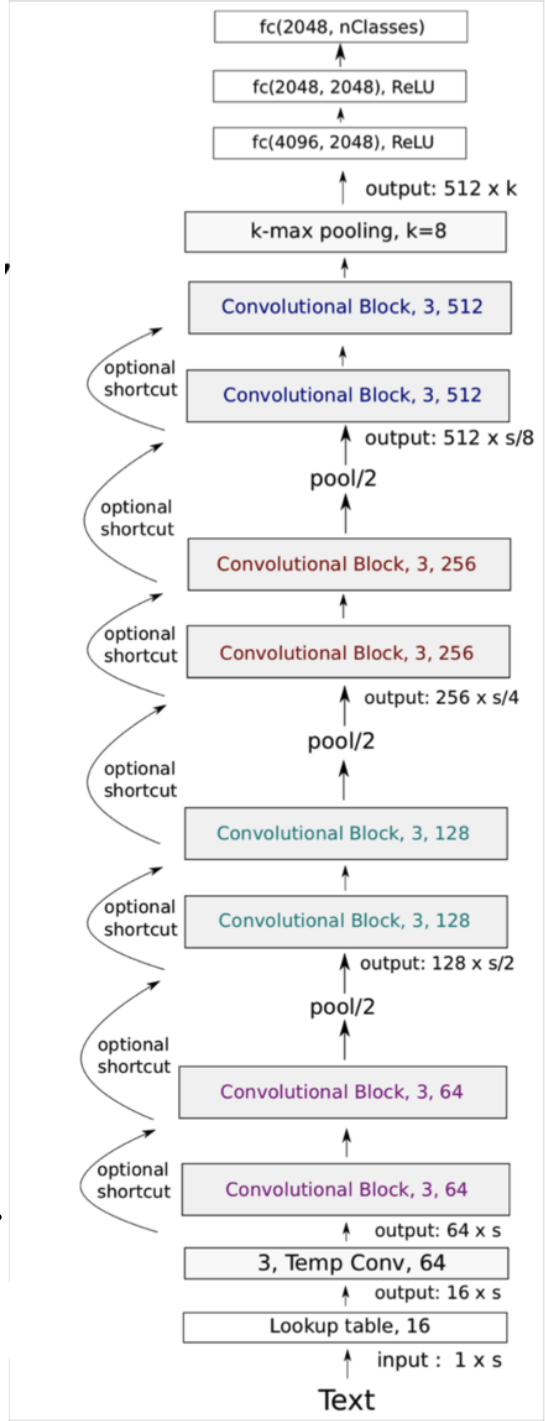


# Very Deep Conv. Nets for Text Classification

- **Traditional NLP Models:**
  - Sequence models (**LSTMs, GRUs**) have dominated NLP.
  - CNNs and **Attention-based models** are also widely used.
  - However, **NLP models have not been as deep** as vision models.
- **What If We Build Vision-Like Deep Networks for NLP?**
  - Inspired by **very deep CNNs in computer vision** (e.g., ResNet, VGG).
  - Works **from the character level** rather than word embeddings.
  - Uses **stacked convolutional layers** to extract hierarchical linguistic features.
- **Advantages:**
  - **Fully convolutional architecture** allows for parallel processing.
  - **Captures deep, multi-scale text representations**.
  - Can outperform traditional sequence models in **text classification** tasks.

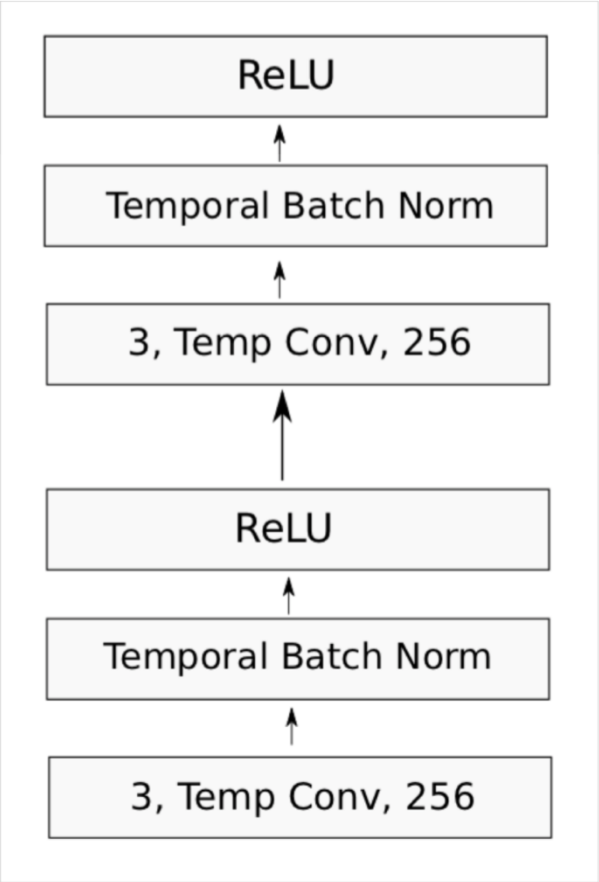
# Very Deep CNN (VD-CNN) Arch.

- **Vision-Inspired Design:**
  - Resembles **VGGNet/ResNet** rather than traditional NLP models.
  - Stacked convolutional layers process text **like images in deep CNNs**.
- **Hierarchical Feature Extraction:**
  - **Local pooling at each stage:**
    - **Halves temporal resolution** (reducing sequence length).
    - **Doubles the number of features** (expanding representation depth).
- **Input and Embeddings:**
  - Input **sequence length (s) = 1024 characters**.
  - **16-dimensional embeddings** for character-level representation.
- **Constant Output Size:**
  - Text is **truncated or padded** to ensure a fixed input size.
  - Enables **efficient processing and scalability** in text classification tasks.



# VD-CNN Blocks: Building Deep Text Representations

- **Structure of Each Block:**
  - **Two convolutional layers** per block.
  - Each convolution is followed by:
    - **Batch Normalization (BatchNorm)** → Stabilizes training.
    - **ReLU Activation** → Introduces non-linearity.
- **Convolutional Properties:**
  - Uses **3×3 convolutional filters** for feature extraction.
  - **Padding ensures spatial preservation** (or halving when pooling).
- **Pooling for Hierarchical Feature Learning:**
  - **Local pooling** reduces temporal resolution.
  - Simultaneously **doubles the number of feature maps**, enhancing representational depth.

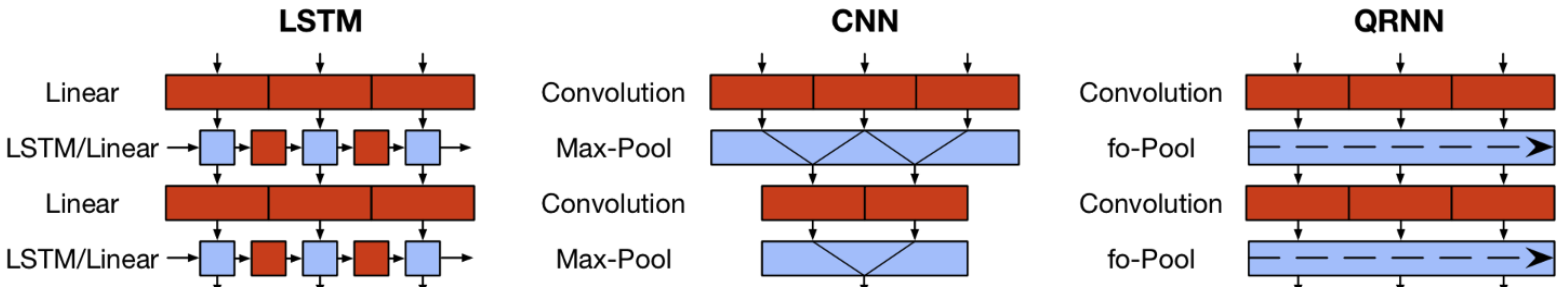


# RNNs Are Very Slow – Need for Alternatives

- **RNNs in Deep NLP:**
  - A **standard building block** for sequence-based tasks (e.g., translation, speech processing).
  - Excellent at capturing **temporal dependencies** but suffer from inefficiencies.
- **Why Are RNNs Slow?**
  - **Poor parallelization:** Each step depends on the previous one.
  - **Long sequences lead to high computational costs.**
  - Training and inference **scale poorly** compared to CNNs.
- **The Solution: Quasi-Recurrent Neural Networks (QRNNs)**
  - Combines the **best of RNNs and CNNs**.
  - Uses **parallelizable convolutional layers** for feature extraction.
  - Replaces recurrent units with a **lightweight gating mechanism**.
  - Achieves **faster training** while maintaining strong sequence modeling capabilities.

# Quasi-Recurrent Neural Networks (QRNNs)

- Tries to combine the best of both worlds




- Convolutions for parallelism across time:

$$\mathbf{z}_t = \tanh(\mathbf{W}_z^1 \mathbf{x}_{t-1} + \mathbf{W}_z^2 \mathbf{x}_t)$$
$$\mathbf{f}_t = \sigma(\mathbf{W}_f^1 \mathbf{x}_{t-1} + \mathbf{W}_f^2 \mathbf{x}_t)$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o^1 \mathbf{x}_{t-1} + \mathbf{W}_o^2 \mathbf{x}_t).$$

→

$$\mathbf{Z} = \tanh(\mathbf{W}_z * \mathbf{X})$$
$$\mathbf{F} = \sigma(\mathbf{W}_f * \mathbf{X})$$
$$\mathbf{O} = \sigma(\mathbf{W}_o * \mathbf{X}),$$

✅ Convolutions compute candidate, forget, and output gates.

$$\mathbf{h}_t = \mathbf{f}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{f}_t) \odot \mathbf{z}_t,$$

- Element-wise gated pseudo-recurrence for parallelism across channels is done in pooling layer:

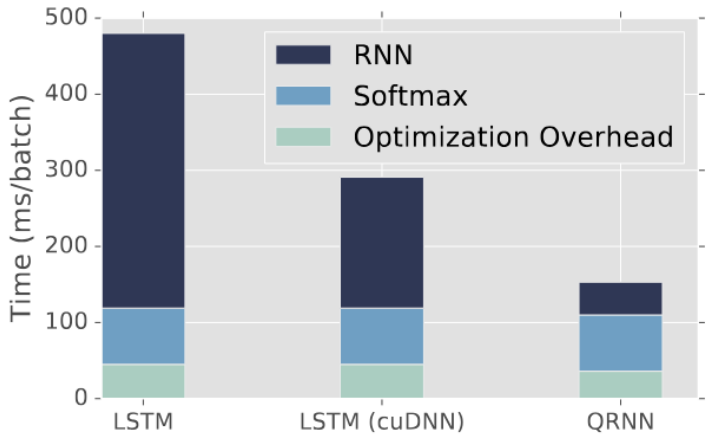
# QRNNs

| Model | Parameters | Validation | Test |
|---|---|---|---|
| LSTM (medium) (Zaremba et al., 2014) | 20M | 86.2 | 82.7 |
| Variational LSTM (medium, MC) (Gal & Ghahramani, 2016) | 20M | 81.9 | 79.7 |
| LSTM with CharCNN embeddings (Kim et al., 2016) | 19M | – | 78.9 |
| Zoneout + Variational LSTM (medium) (Merity et al., 2016) | 20M | 84.4 | 80.6 |
| *Our models* | | | |
| LSTM (medium) | 20M | 85.7 | 82.0 |
| QRNN (medium) | 18M | 82.9 | 79.9 |
| QRNN + zoneout ($p = 0.1$) (medium) | 18M | 82.1 | 78.3 |

✅ Faster

Table 2: Single model perplexity on validation and test sets for the Penn Treebank language modeling task. Lower is better. "Medium" refers to a two-layer network with 640 or 650 hidden units per layer. All QRNN models include dropout of 0.5 on embeddings and between layers. MC refers to Monte Carlo dropout averaging at test time.

- James Bradbury, Stephen Merity, Caiming Xiong, Richard Socher, Quasi-Recurrent Neural Networks, ICLR 2017. https://arxiv.org/abs/1611.01576




| | | Sequence length | | | | |
|---|---|---|---|---|---|---|
| | | 32 | 64 | 128 | 256 | 512 |
| Batch size | 8 | **5.5x** | **8.8x** | **11.0x** | **12.4x** | **16.9x** |
| | 16 | **5.5x** | **6.7x** | **7.8x** | **8.3x** | **10.8x** |
| | 32 | **4.2x** | **4.5x** | **4.9x** | **4.9x** | **6.4x** |
| | 64 | **3.0x** | **3.0x** | **3.0x** | **3.0x** | **3.7x** |
| | 128 | **2.1x** | **1.9x** | **2.0x** | **2.0x** | **2.4x** |
| | 256 | **1.4x** | **1.4x** | **1.3x** | **1.3x** | **1.3x** |

✅ Better

Figure 4: *Left:* Training speed for two-layer 640-unit PTB LM on a batch of 20 examples of 105 timesteps. "RNN" and "softmax" include the forward and backward times, while "optimization overhead" includes gradient clipping, $L^2$ regularization, and SGD computations.
*Right:* Inference speed advantage of a 320-unit QRNN layer alone over an equal-sized cuDNN LSTM layer for data with the given batch size and sequence length. Training results are similar.

# Limitations of Quasi-Recurrent Neural Networks (QRNNs)

- **Struggles with Character-Level Language Models**
  - QRNNs **do not perform as well as LSTMs** on character-based tasks.
  - Likely due to their reduced ability to capture **fine-grained sequential dependencies**.
- **Challenges with Long-Term Dependencies**
  - **Difficulty in modeling very long-range dependencies** compared to true recurrent models.
  - May require **explicit mechanisms** like attention to compensate.
- **Depth vs. Recurrence Trade-off**
  - QRNNs often need **deeper architectures** to match LSTM performance.
  - **However, they remain faster**, even at greater depth.
  - **Key Insight:** They rely on **depth as a substitute for recurrence**, rather than explicit sequential processing.

# Problems with RNNs: Why Move Beyond?

- **Inherent Sequential Processing**
  - RNNs **do not parallelize well** – each step depends on the previous one.
  - Slower training and inference compared to CNNs and Transformer models.
- **Struggles with Long-Range Dependencies**
  - Despite **GRUs and LSTMs**, RNNs still face challenges in capturing distant relationships.
  - The **path length between states increases** with sequence length, leading to **vanishing gradients**.
- **The Role of Attention Mechanisms**
  - Attention allows direct access to **any state in the sequence**, bypassing long chains of dependencies.
  - **Key Question:** If attention provides this access efficiently... **do we even need RNNs anymore?**
  - **Motivation for Transformers** – Replaces recurrence with self-attention for fully parallelized sequence modeling.